DOCUMENTATION REPORT: ACTIVITY 3

System Development, Repository Deployment, and Progress Documentation

Lazarte, Andrei Miguel J.
Opina, Kisha Mae N.
Quintab, Rainne T.

Bachelor of Science in Information Technology
COMP 018: Database Administration
Prof. John Kenneth L. Timblaco

Project Title: HealthBridge Clinic Management System
Group Number: Group 04 (the repository and database use the prefix GRP03 / grp03)
Course & Section: COMP 018 - Database Administration
Database Name: grp03_projectdb (PostgreSQL 17)
Tech Stack: HTML / CSS / JavaScript front end; PHP 8 JSON API on Apache (XAMPP); PostgreSQL 17

Repository Link: https://github.com/RainneQuintab/GRP03_PROJECTDB

Google Sheets Tracker Link: https://docs.google.com/spreadsheets/d/1BClCa9rEs9o-D9Rp3tUZVhECipJ234zX/edit?gid=542211697#gid=542211697

Cover and Team Information

Team Member	Assigned Role	Responsibilities
Quintab	Project Manager / Lead DB Administrator	Oversees project milestones, repository coordination, and PostgreSQL DDL schema design.
Quintab	Lead Front-end Developer	Implements responsive UI components, patient portals, and staff dashboard interfaces.
Lazarte	UI/UX & Workflow Designer	Maps out user role permissions, wireframes, and architecture interaction flows.
Quintab	Lead Back-end Developer	Builds REST API endpoints, routing logic, and database integration drivers.
Opiña	Documentation & QA Lead	Compiles progress logs, verifies CRUD operations, and manages README/PDF documentation.

System Overview

Problem Statement
Many local medical clinics still rely on manual record-keeping or fragmented messaging tools to handle patient queues, doctor schedules, and appointment bookings. This often leads to double-booking, lost medical histories, mismanaged prescription records, and long waiting times for patients.

General Objective & Scope
HealthBridge is a centralized database application that streamlines clinic operations, automates appointment scheduling and queue management, and keeps patient histories, prescriptions and billing in organized digital storage.

- **Included Features:** patient registration and portal, appointment requests, staff assignment of doctor and time slot, doctor consultation notes and prescriptions, billing and online payment records, password-reset workflow, and an admin backup of deleted records with restore. .

- **Excluded Features:** publicly exposed doctor calendars, real payment-gateway integration and real-time video consultation.

User Roles and Database Mapping

Application roles describe what a person can do after logging in. Database roles describe what a PostgreSQL account can do. End users have no PostgreSQL login: the backend always connects through the runtime account hb_app, which inherits hb_staff. The roles are created by database/roles.sql.

Application Role	Responsibilities in Application	Functions and Data Access	Intended Database Role
Administrator	Oversees the clinic and the deleted-records backup	Everything Staff can do, plus view the Backup, restore deleted records or delete them permanently; dashboard overview and recent activity.	hb_admin (elevated group role, not a superuser
Staff / Receptionist	Reviews requests and runs the daily queue.	Review requests, assign doctor and slot, cancel appointments, edit patients, add bills and record payments, delete Completed/Cancelled records (backed up first).	hb_staff
Doctor	Conducts consultations.	See own queue only; complete a visit with diagnosis and prescriptions.	hb_staff
Patient	Manages personal demographics and booking requests.	Register profile, submit appointment requests, and view personal medical history.	app_runtime_grp
Application Runtime	System account used by backend application to connect to PostgreSQL.	Only the queries the backend needs	hb_app (member of hb_staff)

Database role	Type	Privileges
hb_admin	Group role	ALL on all tables and sequences in schema public (and default privileges on future tables).

hb_staff	Group role	SELECT/INSERT/UPDATE/DELETE on patients, appointments, medical_notes, prescriptions, billing and deleted_records; SELECT only on doctors and specialties; SELECT/UPDATE on staff_users; SELECT/INSERT/UPDATE on both password-reset tables; sequence usage.
hb_app	Login (runtime)	Member of hb_staff. Not a superuser. Password is set by the group when running roles.sql (placeholder CHANGE_ME in the repository).

Admin-only actions (view Backup, restore, delete forever) are currently enforced in the PHP layer; Activity 4 can give admin sessions their own database login.

Development Progress Log

Summary of Development Milestones

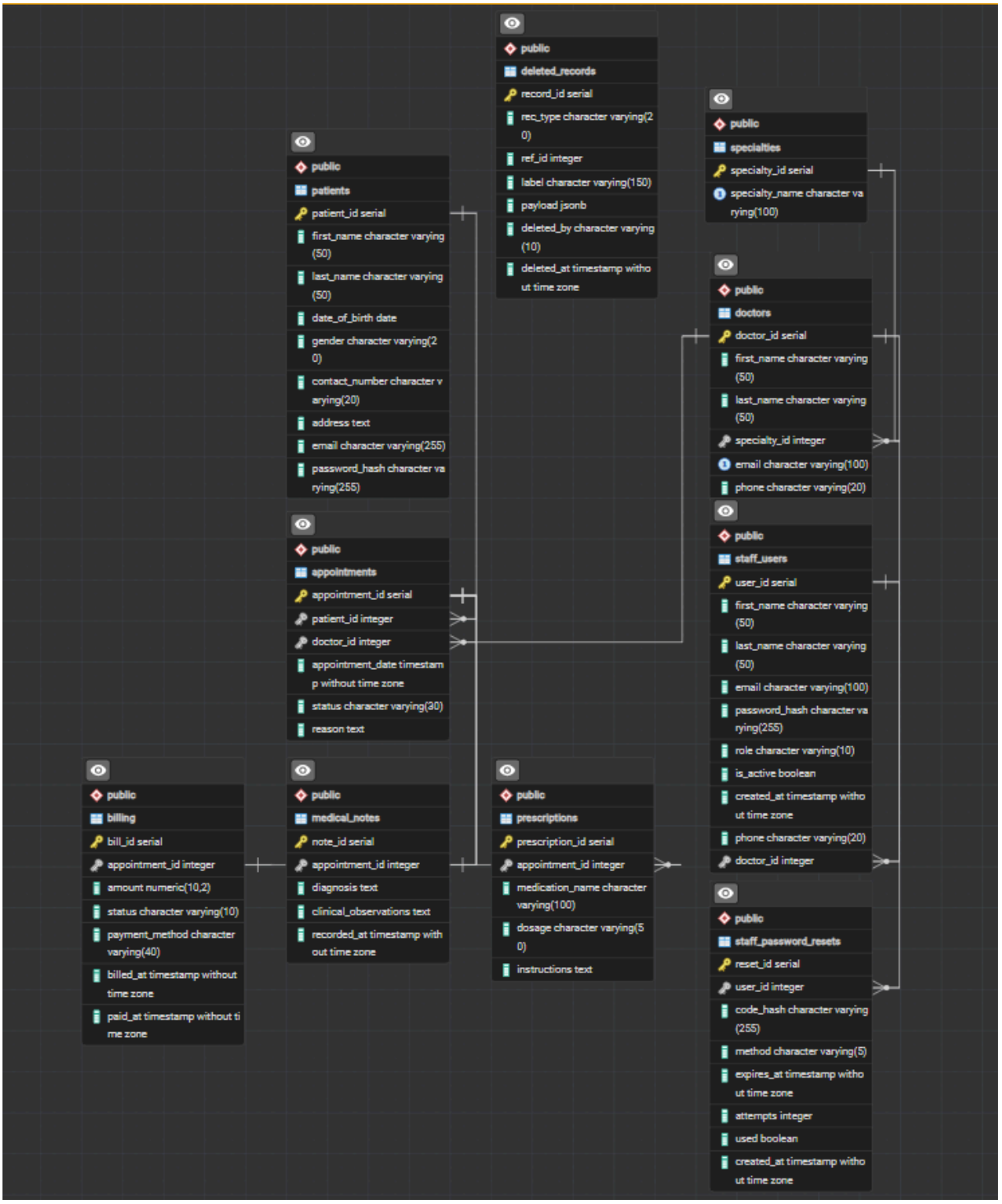
Date	Milestone / Task Completed	Team Member(s) Responsible	Key commits / PRs	Status / Remarks
Week 1	PHP endpoints for login, staff login/logout/session, password reset, doctor list and patient sign-up.	Quintab	Update login.php, staff_login.php , staff_session.php, staff_reset.php, get_doctors.php	Completed
Week 1	Appointment endpoints and patient session check; README revised with tech stack and structure.	Quintab	PR #1 appointments-update; Revise README	Completed
Week 1	Patient booking saved to the database; appointment reason; booked time slots disabled.	Quintab	PR #2 db-appointments	Completed
Week 1	Staff and doctor pages on the database; diagnosis and prescriptions saved; real delete with backup and restore; demo accounts .	Quintab	PR #2 db-appointments	Completed
Week 2	Billing table; delete and backup include bills; one visit note per appointment; patient page shows diagnosis and prescriptions.	Quintab	PR #3 final-touches	Completed

Week 2	Staff billing tab; patient payments saved; patient page Visit records and Billing tabs.	Quintab	PR #4	Completed
Week 2	Review fixes: shared session with SameSite cookie and cross-site POST check; lowercase page filenames; pay several bills in one transaction; reject a zero fee.	Quintab	PR #5 review-fixes (Fix 1 to 4)	Completed
Week 2	README note on script order; .env.example aligned with the real database and user names; documentation compiled.	Quintab	PR #6; Update DB_NAME / DB_USER in .env.example	Completed

Database Implementation (grp03_projectdb)

The database has 11 tables and 2 views in PostgreSQL 17 (schema public, extension pgcrypto). The ERD below was generated from the final schema scripts**.**

Figure 1. Entity-Relationship Diagram (final schema)



Schema Summary

Database Tables Summary

Table	Purpose	Keys and constraints
specialties	Lookup of medical fields.	PK specialty_id; specialty_name UNIQUE.
doctors	Doctor profiles.	PK; FK specialty_id; email UNIQUE.
patients	Demographics and portal login.	PK; unique index on LOWER(email).
appointments	Bookings between patients and doctors.	PK; FKs patient_id, doctor_id; partial unique index (doctor_id, appointment_date) WHERE status <> 'Cancelled' prevents double-booking; reason column; status Pending / Scheduled / Completed / Cancelled.
medical_notes	Diagnosis and observations per visit.	PK; FK appointment_id; one note per appointment.
prescriptions	Medications per visit.	PK; FK appointment_id.
billing	Bills and payments.	PK; appointment_id UNIQUE FK; amount >= 0; status Unpaid/Paid; paid_at set only when Paid.
staff_users	Admin, staff and doctor logins.	PK; role IN (admin, staff, doctor); doctor role requires doctor_id; unique email, phone, doctor.
staff_password_resets	Staff reset codes (hashed).	PK; FK user_id ON DELETE CASCADE.
patient_password_resets	Patient reset codes (hashed).	PK; FK patient_id ON DELETE CASCADE.
deleted_records	Backup copy of deleted patients and appointments.	PK; rec_type IN (patient, appointment); payload JSONB.

Views: appointment_details and visit_records (create_views.sql).

Changes from the approved design (with reasons)

- Added billing, staff_users, staff_password_resets, patient_password_resets and deleted_records to support payments, staff/doctor/admin logins, password resets and the delete-backup feature.
- Added appointments.reason and the status Pending so patients can request a visit before staff assign a doctor.
- appointment_date is a TIMESTAMP (date and slot together) so the partial unique index can block double-booking.

SQL DDL Implementation Scripts

```
None
-- HealthBridge (grp03_projectdb) - all SQL scripts from the database/ folder.
-- RUN ORDER on a new database: schema, seed, roles, update_reason, update_note_unique,
update_billing, create_views, demo_accounts.
-- The last three files (update_appointments, update_backup, update_patient_reset) are
UPGRADE-ONLY: not needed on a new database.
-- Replace CHANGE_ME in roles.sql with your own password before running.

--
==========================================================================
=
-- FILE: database/schema.sql
-- HealthBridge Clinic Management System - Schema
-- Database: grp03_projectdb (PostgreSQL)
-- Run order on an EMPTY database:  schema.sql -> seed.sql -> roles.sql

CREATE EXTENSION IF NOT EXISTS pgcrypto;   -- used by seed.sql to hash starter
passwords

CREATE TABLE specialties (
    specialty_id   SERIAL PRIMARY KEY,
    specialty_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE doctors (
    doctor_id    SERIAL PRIMARY KEY,
    first_name   VARCHAR(50)  NOT NULL,
    last_name    VARCHAR(50)  NOT NULL,
    specialty_id INT          NOT NULL REFERENCES specialties(specialty_id),
    email        VARCHAR(100) UNIQUE NOT NULL,
    phone        VARCHAR(20)
);

CREATE TABLE patients (
    patient_id     SERIAL PRIMARY KEY,
    first_name     VARCHAR(50) NOT NULL,
    last_name      VARCHAR(50) NOT NULL,
    date_of_birth  DATE        NOT NULL,
    gender         VARCHAR(20),
    contact_number VARCHAR(20),
    address        TEXT,
    email          VARCHAR(100),           -- portal login (NULL for walk-in patients)
    password_hash  VARCHAR(255)            -- bcrypt hash from password_hash(); NULL until
the patient creates a password
);
```

```
-- one portal account per email (case-insensitive); NULL emails allowed
CREATE UNIQUE INDEX patients_email_unique ON patients (LOWER(email));

CREATE TABLE appointments (
   appointment_id   SERIAL PRIMARY KEY,
   patient_id       INT       NOT NULL REFERENCES patients(patient_id),
   doctor_id        INT       NOT NULL REFERENCES doctors(doctor_id),
   appointment_date TIMESTAMP NOT NULL,
   status           VARCHAR(30) DEFAULT 'Scheduled'      -- Pending | Scheduled | Completed |
Cancelled
);
-- a doctor cannot have two active appointments at the same time (cancelled ones free the slot)
CREATE UNIQUE INDEX appointments_doctor_slot ON appointments (doctor_id,
appointment_date) WHERE status <> 'Cancelled';

CREATE TABLE medical_notes (
   note_id              SERIAL PRIMARY KEY,
   appointment_id       INT  NOT NULL REFERENCES appointments(appointment_id),
   diagnosis            TEXT NOT NULL,
   clinical_observations TEXT,
   recorded_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE prescriptions (
   prescription_id SERIAL PRIMARY KEY,
   appointment_id  INT          NOT NULL REFERENCES appointments(appointment_id),
   medication_name VARCHAR(100) NOT NULL,
   dosage          VARCHAR(50)  NOT NULL,
   instructions    TEXT         NOT NULL
);

-- ---------- staff / admin / doctor logins (created by the clinic, no public signup) ----------
CREATE TABLE staff_users (
   user_id       SERIAL PRIMARY KEY,
   first_name    VARCHAR(50)  NOT NULL,
   last_name     VARCHAR(50)  NOT NULL,
   email         VARCHAR(100) NOT NULL,
   password_hash VARCHAR(255) NOT NULL,
   role          VARCHAR(10)  NOT NULL CHECK (role IN ('admin', 'staff', 'doctor')),
   doctor_id     INT REFERENCES doctors(doctor_id),     -- required when role = 'doctor'
   phone         VARCHAR(20),                           -- e.g. 09171234567, for SMS reset codes
   is_active     BOOLEAN      NOT NULL DEFAULT TRUE,
   created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT staff_users_doctor_link CHECK (role <> 'doctor' OR doctor_id IS NOT
NULL)
```

```
);
CREATE UNIQUE INDEX staff_users_email_unique  ON staff_users (LOWER(email));
CREATE UNIQUE INDEX staff_users_phone_unique  ON staff_users (phone)     WHERE
phone IS NOT NULL;
CREATE UNIQUE INDEX staff_users_doctor_unique ON staff_users (doctor_id) WHERE
doctor_id IS NOT NULL;

-- one row per password-reset code sent (only a hash of the code is stored)
CREATE TABLE staff_password_resets (
   reset_id   SERIAL PRIMARY KEY,
   user_id    INT         NOT NULL REFERENCES staff_users(user_id) ON DELETE
CASCADE,
   code_hash  VARCHAR(255) NOT NULL,
   method     VARCHAR(5)   NOT NULL CHECK (method IN ('email', 'phone')),
   expires_at TIMESTAMP    NOT NULL,
   attempts   INT          NOT NULL DEFAULT 0,
   used       BOOLEAN      NOT NULL DEFAULT FALSE,
   created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX staff_password_resets_user_idx ON staff_password_resets (user_id, created_at
DESC);

-- one row per patient password-reset code sent (only a hash of the code is stored)
CREATE TABLE patient_password_resets (
   reset_id   SERIAL PRIMARY KEY,
   patient_id INT          NOT NULL REFERENCES patients(patient_id) ON DELETE
CASCADE,
   code_hash  VARCHAR(255) NOT NULL,
   method     VARCHAR(5)   NOT NULL CHECK (method IN ('email', 'phone')),
   expires_at TIMESTAMP    NOT NULL,
   attempts   INT          NOT NULL DEFAULT 0,
   used       BOOLEAN      NOT NULL DEFAULT FALSE,
   created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE INDEX patient_password_resets_idx ON patient_password_resets (patient_id,
created_at DESC);

-- backup copy of deleted patients / appointments (used by backend/staff_delete.php and
backup.php)
CREATE TABLE deleted_records (
   record_id  SERIAL PRIMARY KEY,
   rec_type   VARCHAR(20)  NOT NULL CHECK (rec_type IN ('patient', 'appointment')),
   ref_id     INT          NOT NULL,
   label      VARCHAR(150) NOT NULL,
   payload    JSONB        NOT NULL,
```

```
    deleted_by VARCHAR(10),
    deleted_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- FILE: database/seed.sql
-- HealthBridge Clinic Management System - Seed Data Script
-- Database: grp03_projectdb (PostgreSQL)

INSERT INTO specialties (specialty_name) VALUES
('General Medicine'),
('Pediatrics'),
('Cardiology'),
('Dermatology');

INSERT INTO doctors (first_name, last_name, specialty_id, email, phone) VALUES
('Maria', 'Santos', 1, 'maria.santos@healthbridge.com', '09171234567'),
('Juan', 'Reyes', 2, 'juan.reyes@healthbridge.com', '09181234567');

-- sample patients can sign in (patient ID HB-00001 / HB-00002 or the email). Sample password:
<demo-password>
INSERT INTO patients (first_name, last_name, date_of_birth, gender, contact_number, address,
email, password_hash) VALUES
('Ana', 'Cruz', '1995-05-12', 'Female', '09223334455', 'San Pedro, Laguna',
'ana.cruz@example.com', crypt('<demo-password>', gen_salt('bf'))),
('Pedro', 'Penduko', '1988-11-20', 'Male', '09334445566', 'Biñan, Laguna',
'pedro.penduko@example.com', crypt('<demo-password>', gen_salt('bf')));

INSERT INTO appointments (patient_id, doctor_id, appointment_date, status) VALUES
(1, 1, '2026-11-10 09:00:00', 'Scheduled'),
(2, 2, '2026-11-10 10:30:00', 'Scheduled');

INSERT INTO medical_notes (appointment_id, diagnosis, clinical_observations) VALUES
(1, 'Viral Infection', 'Patient presents with mild fever and cough.'),
(2, 'Routine Pediatric Checkup', 'Child is growing normally with healthy vitals.');

INSERT INTO prescriptions (appointment_id, medication_name, dosage, instructions) VALUES
(1, 'Paracetamol', '500mg', 'Take 1 tablet every 4 hours as needed for fever.'),
(2, 'Multivitamins Syrup', '5ml', 'Take once daily in the morning.');

-- ---------- starter logins (needed to sign in the first time) ----------
-- !! These passwords are public in this repo. Change them right after the first login. !!
INSERT INTO staff_users (first_name, last_name, email, password_hash, role) VALUES
 ('Clinic', 'Admin', 'admin@healthbridge.com', crypt('<demo-password>', gen_salt('bf')), 'admin'),
 ('Front',  'Desk',  'staff@healthbridge.com', crypt('<demo-password>', gen_salt('bf')), 'staff');
```

```
-- one doctor login per doctor (email = doctor's email, password <demo-password>)
INSERT INTO staff_users (first_name, last_name, email, password_hash, role, doctor_id, phone)
SELECT d.first_name, d.last_name, d.email, crypt('<demo-password>', gen_salt('bf')), 'doctor',
d.doctor_id, d.phone
FROM doctors d;

-- FILE: database/roles.sql
-- Database access control. Run AFTER schema.sql and seed.sql.
-- Replace CHANGE_ME with a strong password, and put the same value in
backend/config.php.

CREATE ROLE hb_admin;
CREATE ROLE hb_staff;
CREATE USER hb_app WITH PASSWORD 'CHANGE_ME';

GRANT CONNECT ON DATABASE grp03_projectdb TO hb_admin, hb_staff, hb_app;
GRANT USAGE ON SCHEMA public TO hb_admin, hb_staff, hb_app;

-- admin: everything
GRANT ALL PRIVILEGES ON ALL TABLES    IN SCHEMA public TO hb_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO hb_admin;

-- staff: clinical data (read/write), reference data (read only)
GRANT SELECT, INSERT, UPDATE ON patients, appointments, medical_notes, prescriptions
TO hb_staff;
GRANT SELECT ON doctors, specialties TO hb_staff;

-- login & password-reset tables used by the PHP backend
GRANT SELECT, UPDATE         ON staff_users           TO hb_staff;
GRANT SELECT, INSERT, UPDATE ON staff_password_resets, patient_password_resets TO
hb_staff;

GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO hb_staff;

-- the PHP backend connects as hb_app, which inherits hb_staff
GRANT hb_staff TO hb_app;

ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT ALL ON TABLES TO
hb_admin;

-- staff may delete clinical records; every delete is backed up first (see deleted_records)
GRANT DELETE ON patients, appointments, medical_notes, prescriptions TO hb_staff;
GRANT SELECT, INSERT, UPDATE, DELETE ON deleted_records TO hb_staff;
GRANT USAGE, SELECT ON SEQUENCE deleted_records_record_id_seq TO hb_staff;
```

```
-- FILE: database/update_reason.sql
ALTER TABLE appointments ADD COLUMN IF NOT EXISTS reason TEXT;

-- FILE: database/update_note_unique.sql
ALTER TABLE medical_notes ADD CONSTRAINT one_note_per_visit UNIQUE
(appointment_id);

-- FILE: database/update_billing.sql
CREATE TABLE IF NOT EXISTS billing (
    bill_id        SERIAL PRIMARY KEY,
    appointment_id INT NOT NULL UNIQUE REFERENCES appointments(appointment_id),
    amount         NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    status         VARCHAR(10) NOT NULL DEFAULT 'Unpaid' CHECK (status IN ('Unpaid',
'Paid')),
    payment_method VARCHAR(40),
    billed_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    paid_at        TIMESTAMP,
    CHECK ((status = 'Paid') = (paid_at IS NOT NULL))
);

DO $$
BEGIN
  IF EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hb_staff') THEN
    GRANT SELECT, INSERT, UPDATE, DELETE ON billing TO hb_staff;
    GRANT USAGE, SELECT ON SEQUENCE billing_bill_id_seq TO hb_staff;
  END IF;
END $$;

-- FILE: database/create_views.sql
CREATE OR REPLACE VIEW appointment_details AS
SELECT a.appointment_id,
       a.appointment_date,
       a.status,
       a.reason,
       p.patient_id,
       p.first_name || ' ' || p.last_name AS patient_name,
       d.doctor_id,
       'Dr. ' || d.first_name || ' ' || d.last_name AS doctor_name,
       s.specialty_name
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
LEFT JOIN specialties s ON s.specialty_id = d.specialty_id
ORDER BY a.appointment_date;
```

```
CREATE OR REPLACE VIEW visit_records AS
SELECT a.appointment_id,
       a.appointment_date,
       p.first_name || ' ' || p.last_name AS patient_name,
       'Dr. ' || d.first_name || ' ' || d.last_name AS doctor_name,
       n.diagnosis,
       n.clinical_observations,
       pr.medication_name,
       pr.dosage,
       pr.instructions
FROM appointments a
JOIN patients p ON p.patient_id = a.patient_id
JOIN doctors d ON d.doctor_id = a.doctor_id
JOIN medical_notes n ON n.appointment_id = a.appointment_id
LEFT JOIN prescriptions pr ON pr.appointment_id = a.appointment_id
ORDER BY a.appointment_date, a.appointment_id;

-- FILE: database/demo_accounts.sql
-- Run LAST on a fresh database. TEST ACCOUNTS ONLY - every password below is:
<demo-password>
-- Safe to run more than once.

UPDATE specialties SET specialty_name = 'General Practice' WHERE specialty_name =
'General Medicine';

-- Dr. Carlo Dizon (Cardiology)
INSERT INTO doctors (first_name, last_name, specialty_id, email, phone)
SELECT 'Carlo', 'Dizon', specialty_id, 'carlo.dizon@healthbridge.com', '09201234567'
FROM specialties WHERE specialty_name = 'Cardiology'
ON CONFLICT (email) DO NOTHING;

INSERT INTO staff_users (first_name, last_name, email, password_hash, role, doctor_id, phone)
SELECT d.first_name, d.last_name, d.email, crypt('<demo-password>', gen_salt('bf')), 'doctor',
d.doctor_id, d.phone
FROM doctors d
WHERE d.email = 'carlo.dizon@healthbridge.com'
  AND NOT EXISTS (SELECT 1 FROM staff_users s WHERE LOWER(s.email) =
LOWER(d.email));

-- one shared test password for every demo login
UPDATE staff_users SET password_hash = crypt('<demo-password>', gen_salt('bf'))
WHERE LOWER(email) IN ('admin@healthbridge.com', 'staff@healthbridge.com',
                       'maria.santos@healthbridge.com', 'juan.reyes@healthbridge.com',
                       'carlo.dizon@healthbridge.com');
```

```
UPDATE patients SET password_hash = crypt('<demo-password>', gen_salt('bf'))
WHERE LOWER(email) IN ('ana.cruz@example.com', 'pedro.penduko@example.com');
```

Seed data: seed.sql loads specialties, doctors, sample patients and starter logins; demo_accounts.sql adds the demo accounts listed in the README (class demo only).

Application Evidence (CRUD Results)

The system provides separate sign-in pages for patients and for staff/admin, with role-based access after login. Passwords are masked in every screenshot.

Operation	Evidence in the running system	Figure
Create	Patient signs in and submits an appointment request; admin activity log shows a bill added.	3, 8, 6
Read	Dashboards, schedule, doctor availability and patient list load live data from PostgreSQL.	5, 7, 12, 13, 15
Update	Staff assigns a doctor and time slot, changing the request from Pending to Scheduled; patient details are editable.	9, 10, 11, 15
Delete	Admin recent activity shows appointments "moved to backup" (copied to deleted_records, then removed).	5, 6

Figure 2. Admin sign-in (staff sign-in page)

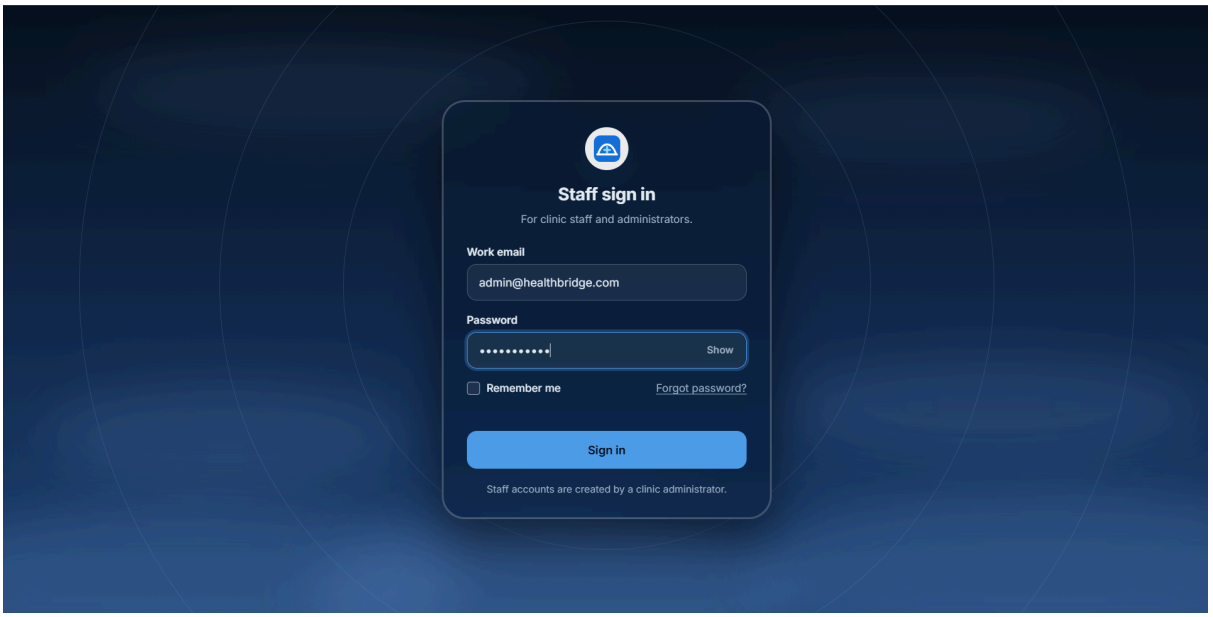


Figure 3. Patient sign-in (with Create an account)

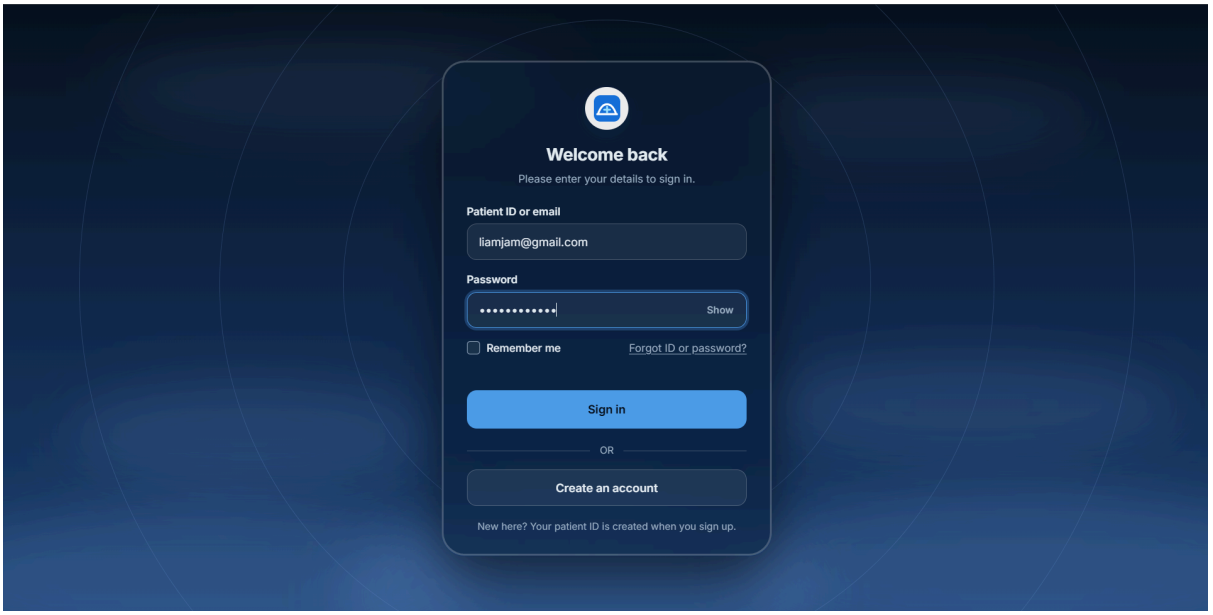
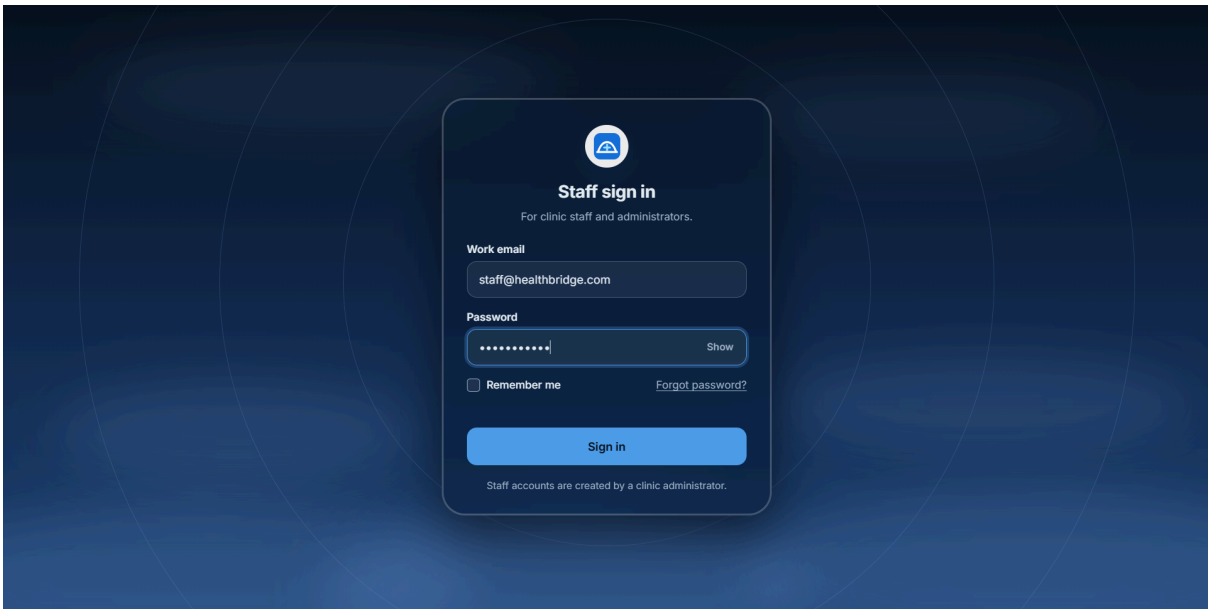


Figure 4. Staff sign-in



The system provides distinct authentication interfaces for **Patients**, **Staff**, and **Admins**, ensuring role-based access control where users securely log in using their credentials or register an account.

Figure 5. Admin dashboard (overview, doctor workload, appointments by status)

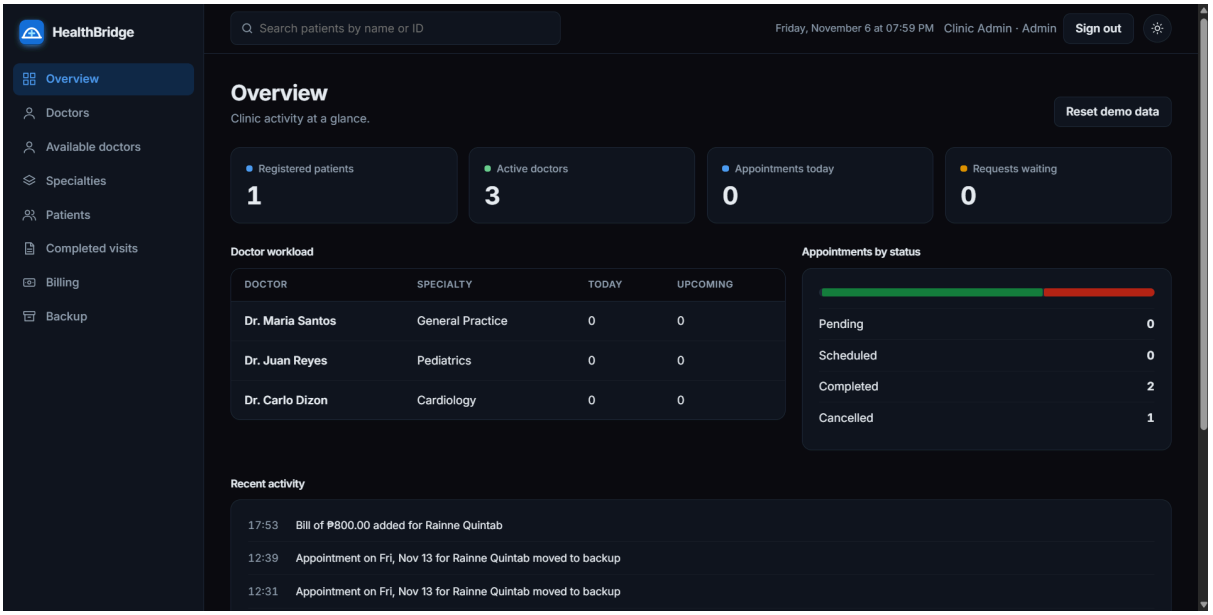
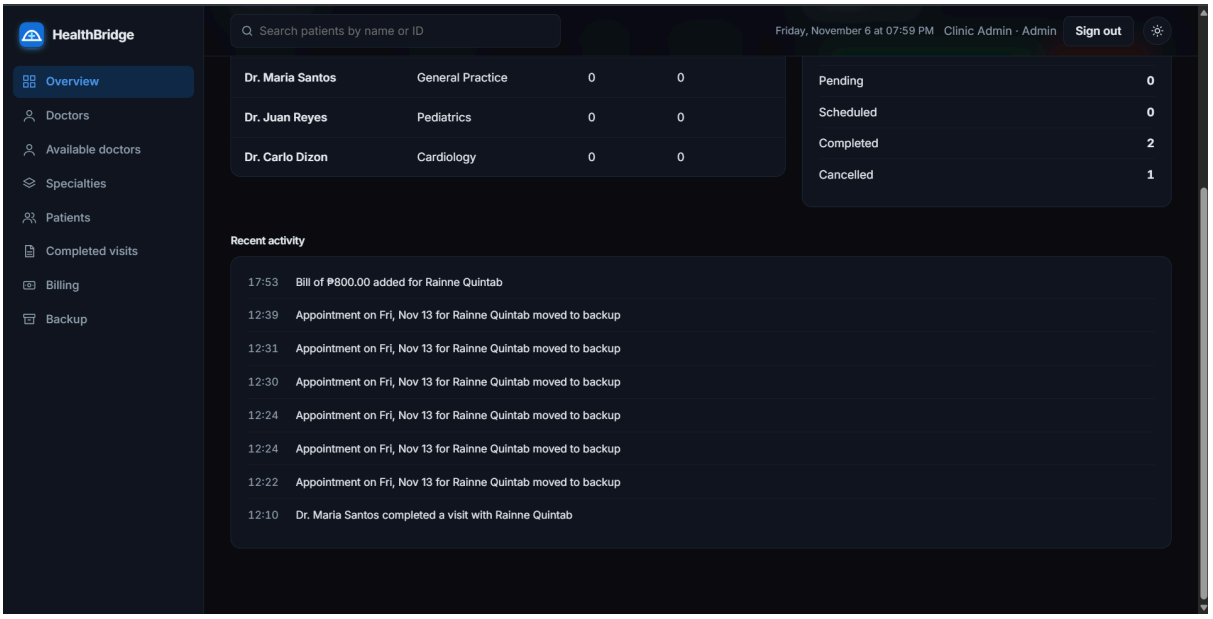


Figure 6. Admin dashboard - recent activity (bill added, appointments moved to backup)



The Clinic Admin dashboard summarizes registered patients, active doctors, daily appointment metrics and doctor workloads, with an activity log of recent system changes.

Figure 7. Patient dashboard (no appointments yet)

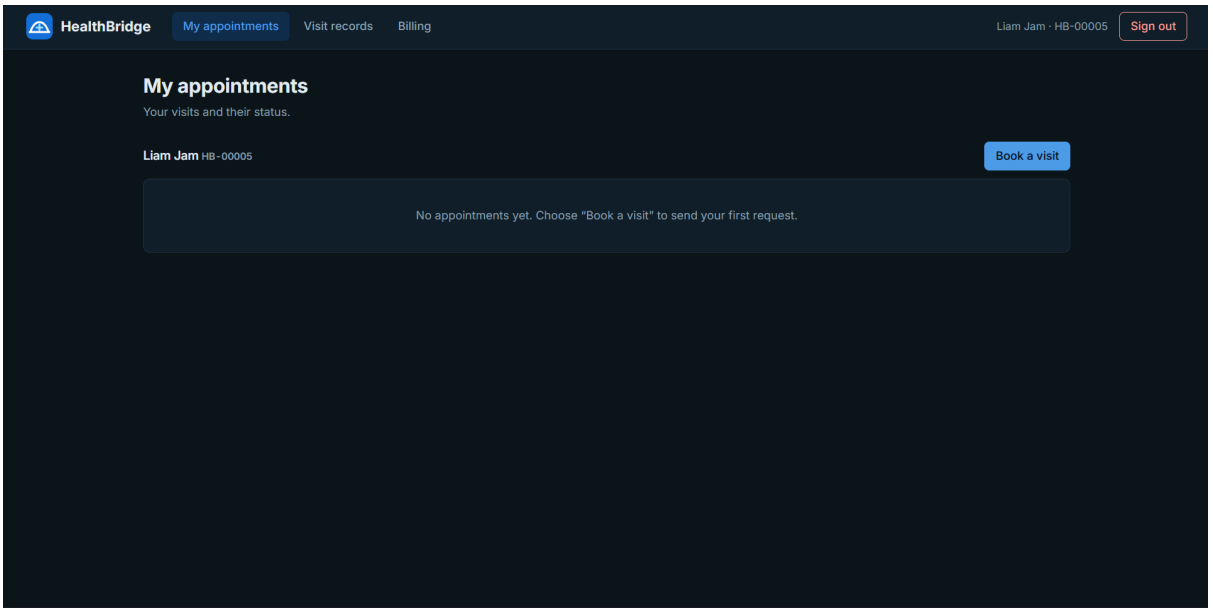
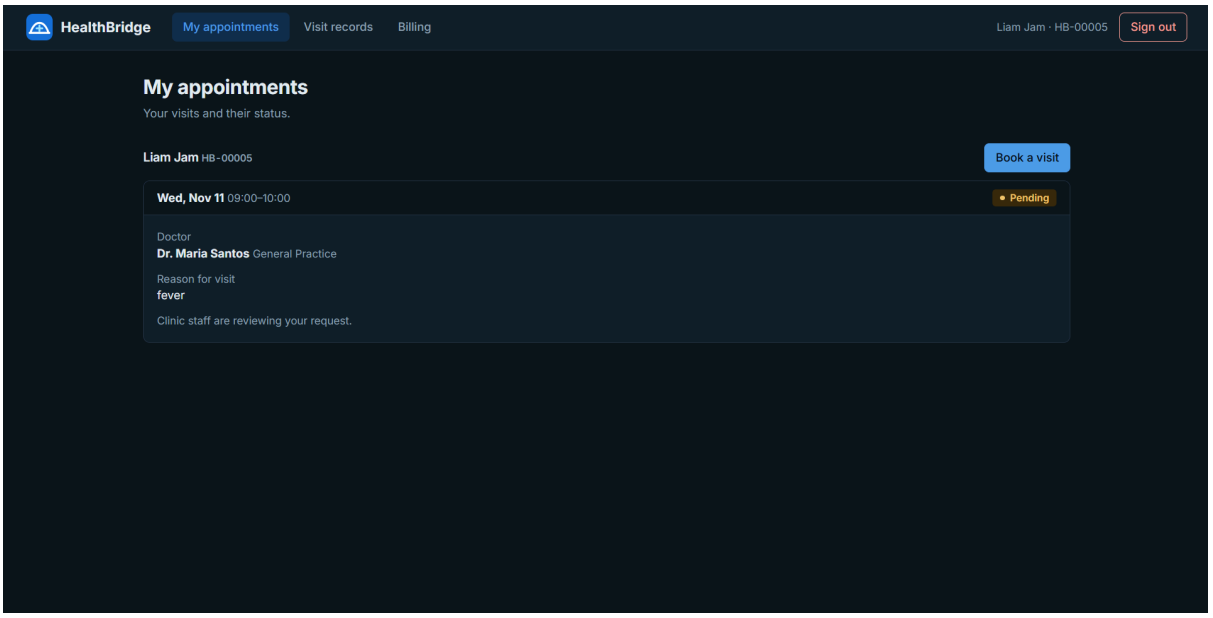


Figure 8. Patient appointment request - shows as Pending for staff review



After signing in, the patient (for example Liam Jam, HB-00005) sees personal appointments and their status. "Book a visit" submits a consultation request with a preferred date and a reason for the visit (here: fever).

Figure 9. Staff dashboard - incoming appointment requests

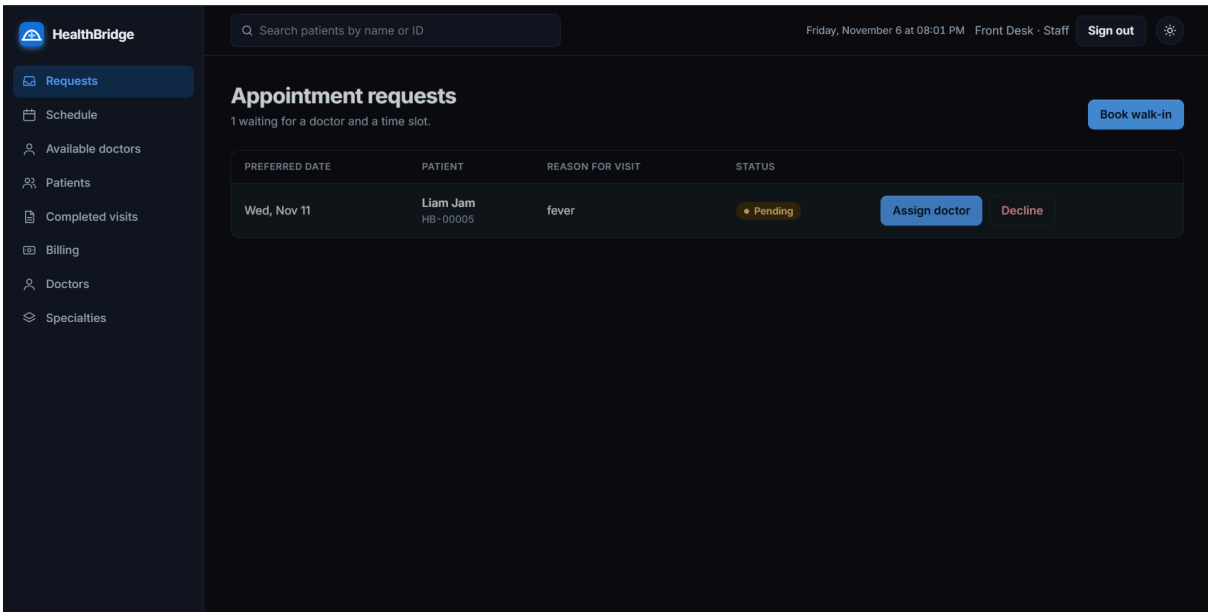
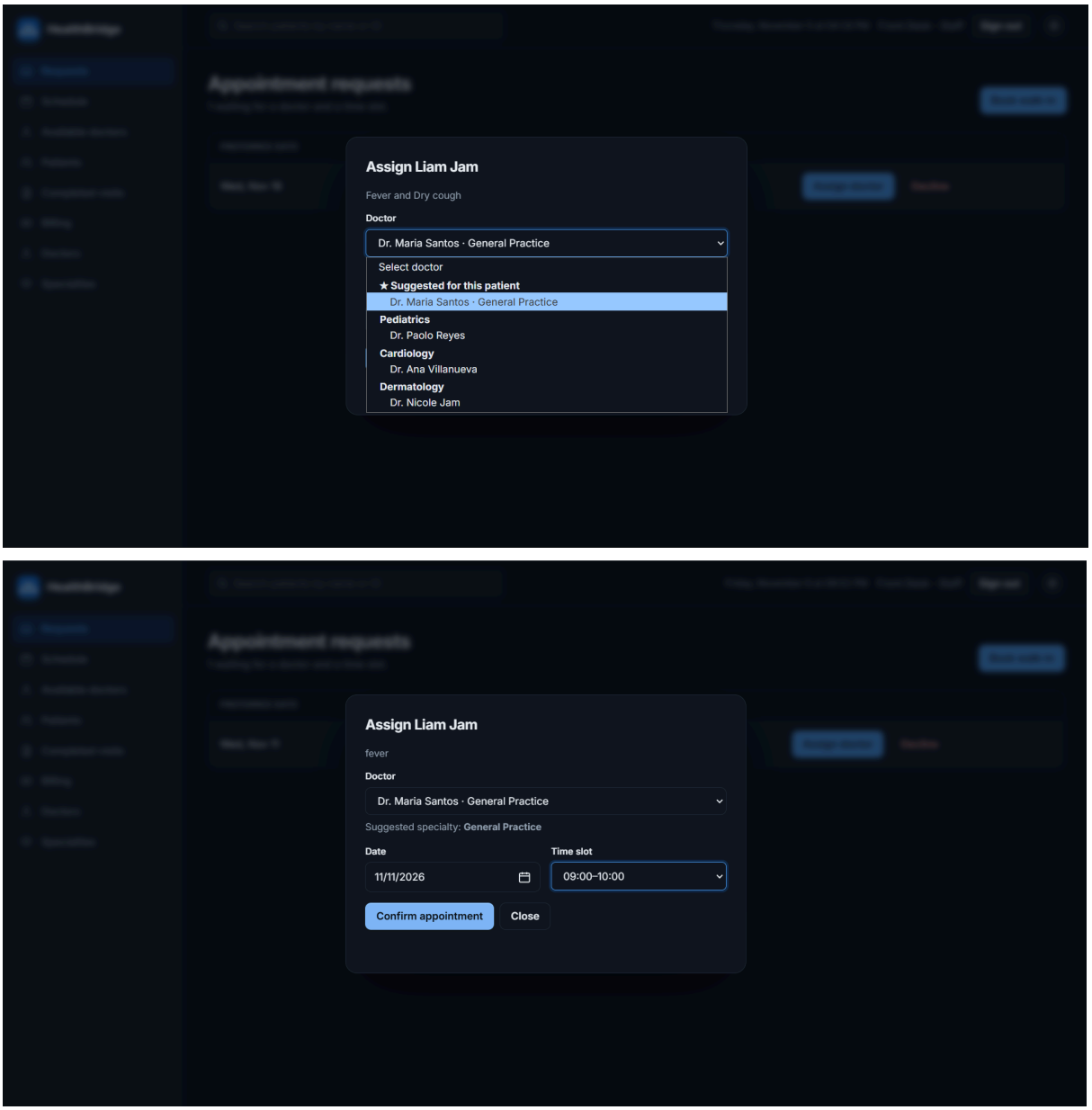


Figure 10. Staff assigns a doctor (suggested by specialty)



Front-desk staff review pending requests, see the suggested specialty, and assign a doctor, date and time slot. The database rejects a second active appointment for the same doctor and time.

Figure 11. Staff schedule view (one column per doctor)

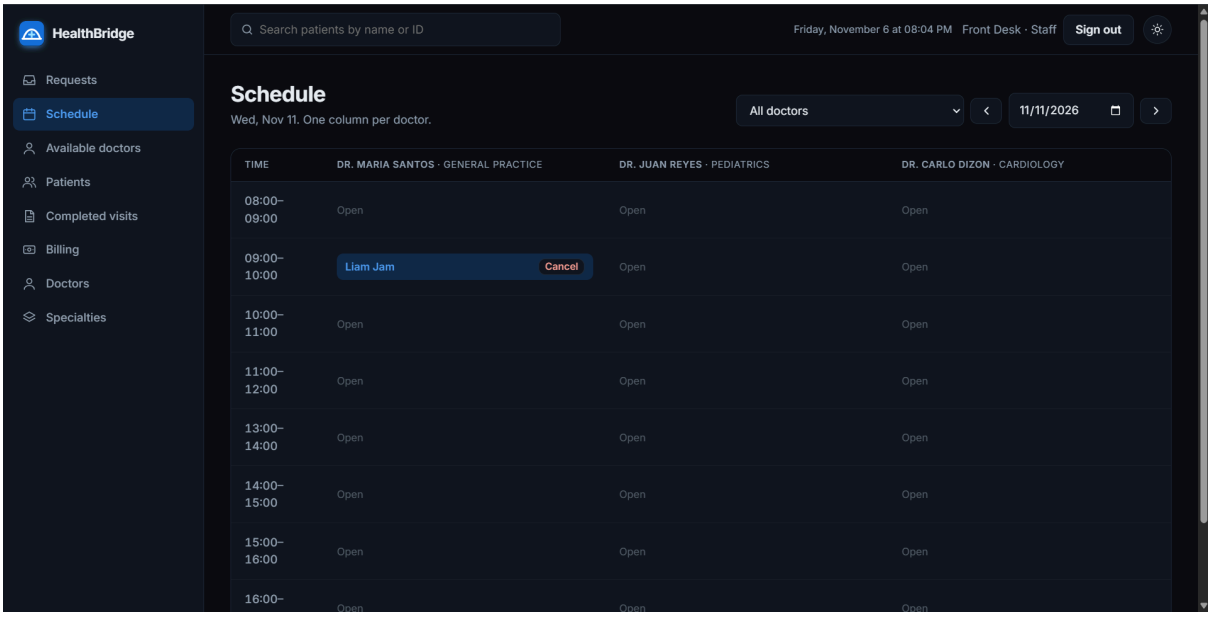


Figure 12. Staff view of doctor availability

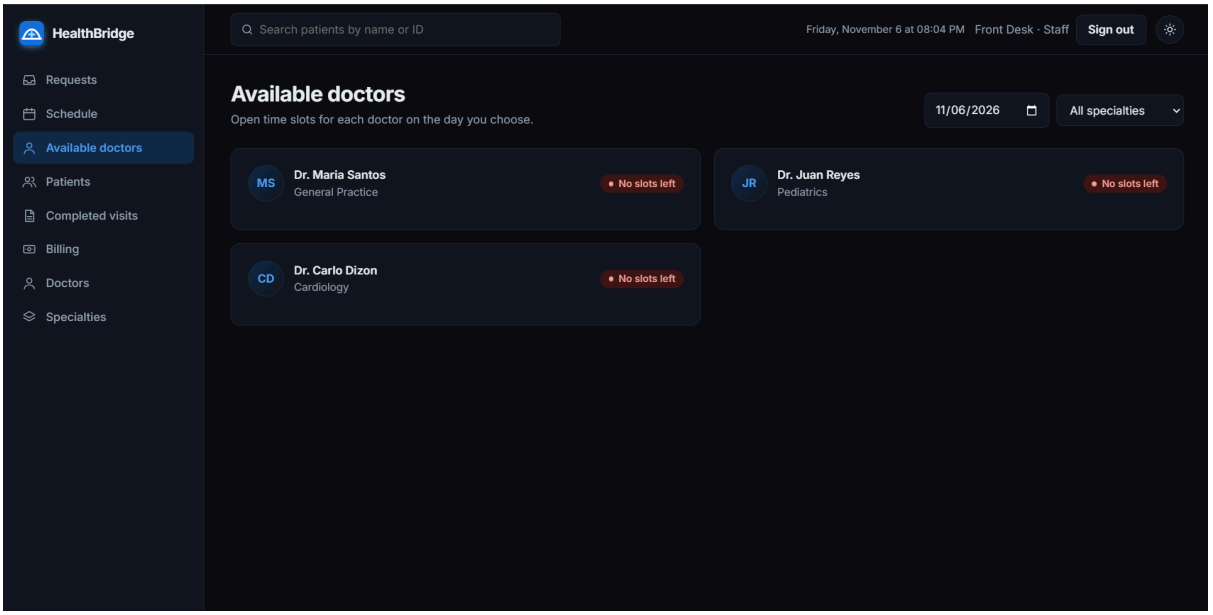
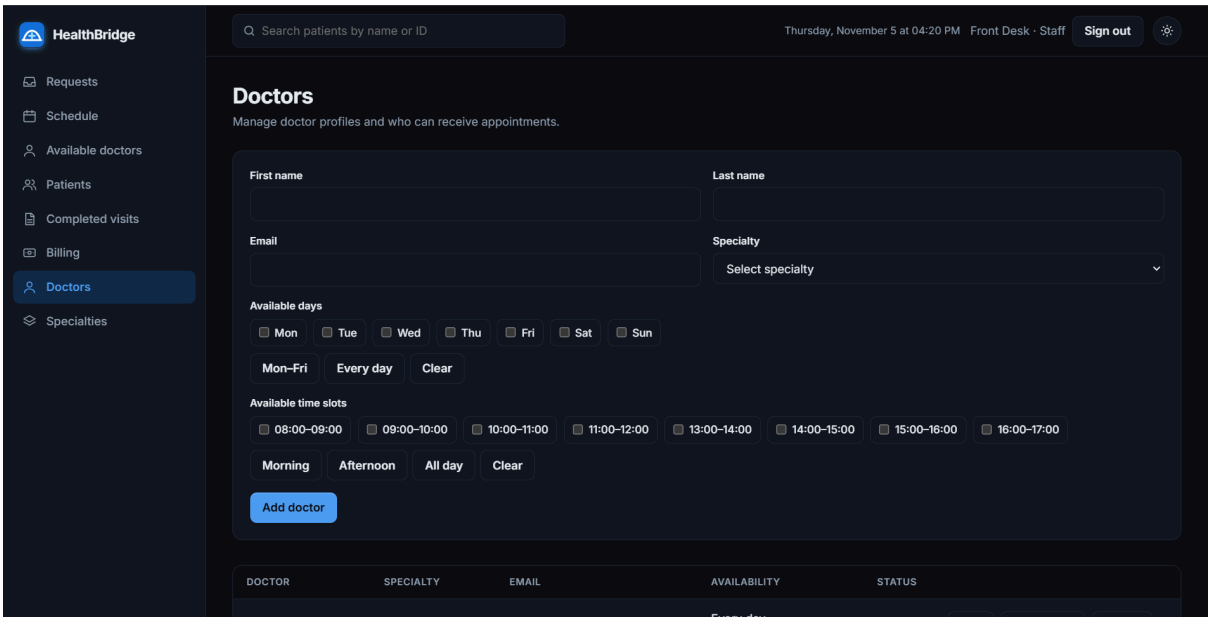
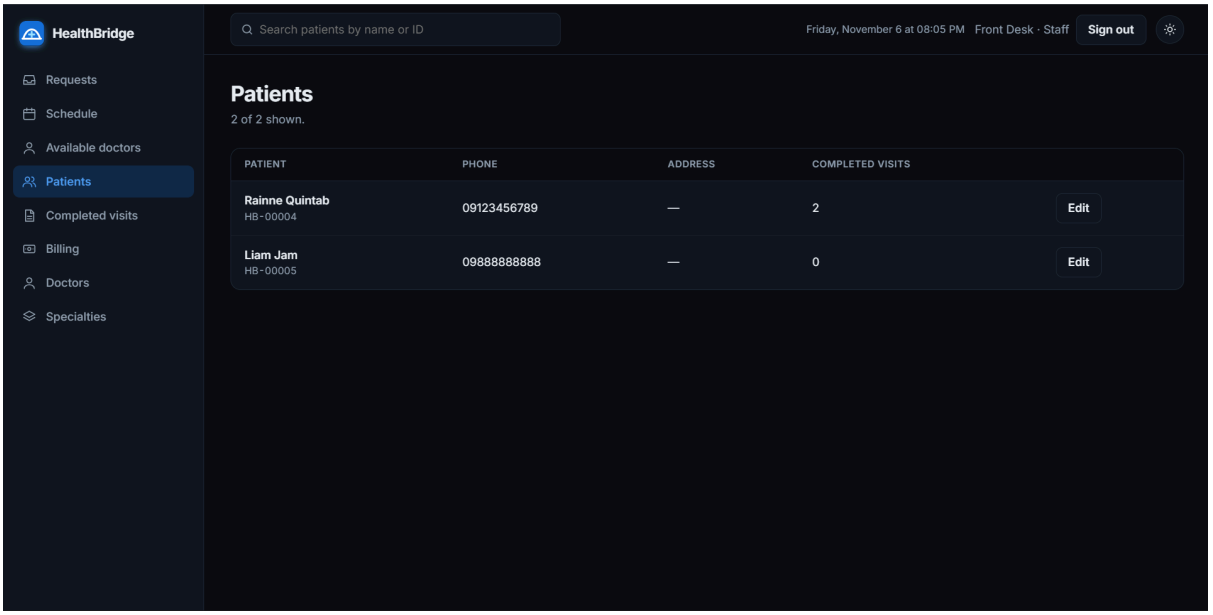


Figure 13. Staff doctor management and availability setup



Staff can monitor open time slots, view schedules across specialties (General Practice, Pediatrics, Cardiology, Dermatology) and manage doctor profiles to balance workload.

Figure 14. Staff patient list (view and edit)



The patient module lets staff view, verify and edit contact numbers, addresses and completed-visit counts.

Setup and Configuration

The PHP backend reads backend/config.php (copied from config.sample.php; git-ignored). .env.example lists the same values for reference. Placeholders only:

Code snippet

```
None
DB_HOST=localhost
DB_PORT=5432
DB_NAME=grp03_projectdb
DB_USER=hb_app
DB_PASSWORD=CHANGE_ME
```

Installation steps

1. Install XAMPP (Apache + PHP 8) and PostgreSQL 17 with pgAdmin. In C:\xampp\php\php.ini enable extension=pgsql and extension=pdo_pgsql, then restart Apache.
2. Put the project in C:\xampp\htdocs\GRP03_PROJECTDB (copy it or run git clone https://github.com/RanneQuintab/GRP03_PROJECTDB there).
3. In pgAdmin create an empty database named grp03_projectdb.
4. Connected as postgres, run in this order: schema.sql, seed.sql, roles.sql (replace CHANGE_ME with your own password first), update_reason.sql, update_note_unique.sql, update_billing.sql, create_views.sql, demo_accounts.sql. Run roles.sql before update_billing.sql so billing is granted to hb_staff.
5. Copy backend/config.sample.php to backend/config.php; set user hb_app and the password from step 4.
6. Start Apache and PostgreSQL and open http://localhost/GRP03_PROJECTDB/frontend/HealthBridge-Login.html (staff, admin and doctors use HealthBridge-StaffLogin.html).

Database user and role setup

Run by roles.sql: CREATE ROLE hb_admin; CREATE ROLE hb_staff; CREATE USER hb_app WITH PASSWORD '...'; GRANT hb_staff TO hb_app; followed by the table privileges listed under User Roles. No real passwords are committed under User Roles. No real passwords are committed.

Issues and Resolutions

Date	Issue / problem	Resolution / action	Status
Week 2	Connection refused when connecting to PostgreSQL locally.	Checked Windows Services and confirmed the postgresql service was running on port 5432.	Resolved
Week 2	Foreign key violation when inserting appointment test data.	Ran parent tables (specialties, doctors) and seed records before bookings.	Resolved
Week 2	Billing table was not granted to hb_staff when update_billing.sql ran before roles.sql.	Documented the order roles.sql then update_billing.sql in the README and troubleshooting guide.	Resolved
Week 2	Admin and staff pages did not open because page filenames differed in letter case from the PHP redirects.	Lowercased the filenames to match the redirects (review Fix 2).	Resolved
Week 2	Session cookies and cross-site form posts were not restricted.	Added a shared session.php with a SameSite cookie and a cross-site POST check (review Fix 1).	Resolved
Week 2	Paying several bills at once could leave a partial result; a fee of 0 was accepted.	Several bills are now paid in one transaction; a zero fee is rejected (review Fix 3 and 4).	Resolved

Cross-Group Verification

Group 04 cloned the other groups' repositories from the tracker and recorded each result in the Activity 3 Testing Log tab of the class Google Sheets tracker. The table summarizes those entries.

Project tested	Setup result	Roles checked	Feedback sent
G01 DropBoxed			
G02 Eventer (HSEMS)			
G03 PAWPAW			

Project tested	Setup result	Roles checked	Feedback sent
G05 SHIORIcafe			
G06 Car Rental Management System			
G07 Applicant Management System			
G08 Catering Management			

Feedback received and fixes. Peer feedback on navigation clarity and minor UI alignment on the patient appointment dashboard was addressed. The testing groups' remaining notes and our resolutions are recorded in the tracker.

AI Usage Declaration

AI Tool Used	Specific Task / Purpose Performed	Validation & Group Corrections
ChatGPT	Drafted initial boilerplate routing structure and system architecture diagrams.	Group reviewed, modified and verified the component tiers against the project specifications.
Gemini	Formatting normalization review sections and data dictionary descriptions.	Group cross-checked all table schemas, constraints and data types against the actual pgAdmin instance.
Claude	Generated responsive HTML5 layout components, CSS styling and front-end templates (frontend/ folder).	Group customized the layouts, integrated form validation and verified form submissions against the backend PHP endpoints.